Software Performance Engineering Course 2025/2026

Performance Modelling of a Microservice Application: Analytical Queueing Network Formulation, Empirical Parameterisation, and Validation

Benchmark System: Google Online Boutique — 11-Service Polyglot Microservice Application

Authors: Essa Janneh | Opoka JM Silvio

Date: August 25th, 2026

https://github.com/Silvio777-hub/Analytical-QN-LQN-Model-of-a-Microservice-Application

Abstract

This report presents an analytical queueing network model for the Google Online Boutique microservice benchmark, parameterised from 6,642 Jaeger telemetry spans across 687 traces collected during an authoritative live load test on October 8, 2026. All ten microservices were measured empirically from Jaeger traces, with the Redis in-memory state store evaluated at its standard 2.0 ms operational baseline. The model identifies the Go Frontend as the primary system bottleneck (S_0 = 18.88 ms, D_0 = 18.88 ms, 18.88% CPU at λ_0 = 10 req/s, 13.43× larger demand than the second-heaviest service, CartService at 1.41 ms, capturing 56.44% of total optimization leverage). Empirical multi-tier validation across four workload tiers ($\lambda \in \{2.0, 5.0, 8.0, 10.0\}$ req/s) reveals an overall Weighted Mean Absolute Percentage Error (WMAPE) of 82.81% and $R^2 = -0.986$. This divergence honestly exposes the fundamental limitation of open queueing networks in shared-host environments: while the analytical model accurately predicts idealized uncontended cloud execution, single-host Docker Desktop resource contention (WSL2 scheduling jitter, NAT bridging, and 16 co-located containers) introduces severe tail inflation and saturates near 5 req/s. Universal Scalability Law (USL) analysis demonstrates that scaling the Frontend to c = 3 replicas achieves 93.6% efficiency, expanding capacity to 148.75 req/s.

Table of Contents

Abstract ii

Table of Contents ii

List of Figures iii

List of Tables iii

Executive Summary & Core Finding iii

1.0 Introduction: Why Microservice Performance is Hard ..... a

2.0 Understanding Queueing Networks (The Math Made Simple) ..... a

3.0 System Architecture & Real-World Tracing Data ..... d

4.0 Measuring Service Times & Traffic Routing ..... f

5.0 Finding the System Bottleneck i

6.0 Model Accuracy & Real-World Validation ..... j

7.0 How to Scale the System: Multi-Replica Analysis l

8.0 Sensitivity Analysis: Where Should Engineers Focus? ..... n

9.0 Sources of Error, Modeling Assumptions & Limitations ..... o

10.0 Conclusion & Actionable Platform Engineering Recommendations ..... o

References ..... q

Appendix A – Comprehensive Mathematical Formulas Reference Matrix ..... q

Appendix B – Summary of Key Benchmark Numbers ..... s
Section 8: Multi-Tier Live Empirical Validation & HPA Autoscaling Policy ..... u
8.1 Multi-Tier Live Empirical Validation (Oct 8, 2026) ..... u
8.2 Mathematical Kubernetes HPA Autoscaling Policy v

List of Figures

TOC \h \z \c "Figure" Figure 1: Reconstructed Microservice Call Graph for Google Online Boutique ..... e
Figure 2: Layered Queueing Network (LQN) Three-Tier Architecture ..... f
Figure 3: Predicted Microservice Utilisation at λ_0 = 10 req/s (Model) 12
Figure 4: Analytical Model Predictions vs. Real-World Benchmark (1 Empirical Tier) 14
Figure 5: Model-Predicted System Latency Curve & 500 ms SLA Boundary 15
Figure 6: Predicted System Response Time Under Multi-Replica Frontend Scaling (Model) 18
Figure 7: Sensitivity Tornado Chart: Where System Speedup Comes From ..... o

List of Tables

TOC \h \z \c "Table" Table 1: Concept, Symbol, Definition, and Everyday Analogy ..... a
Table 2: Open Jackson QN vs. Layered Queueing Network (LQN) Architecture c
Table 3: Queueing Model Foundational Parameters c
Table 4: Online Boutique Microservices Runtime, Protocol, and Architectural Role ..... d
Table 5: Visit Ratio, Exclusive Service Time, Total Demand, and CPU Load at 10 req/s ..... g
Table 6: Empirical Service Times Breakdown (Mean, Median, P95, P99) ..... h
Table 7: Model Predictions vs. Observed Response Times (1 Empirical Tier) 13
Table 8: M/M/c Horizontal Pod Autoscaling Performance & USL Efficiency ..... m
Table 9: Sensitivity Analysis & Engineering Optimization Priority Ranking ..... n
Table 10: Comprehensive Mathematical Formulas Reference Matrix c
Table 11: Architectural and Operational Benchmark Constants ..... e

Executive Summary & Core Finding

We constructed an analytical performance model of the Google Online Boutique microservice benchmark (11 polyglot services), parameterised from 6,642 telemetry spans across 687 traces collected during an authoritative live benchmark on October 8, 2026. The model predicts uncontended response times, queue lengths, and saturation bounds in under 50 ms, providing an asymptotic performance baseline for cloud deployment.

Three Key Architectural Findings

The Frontend service demand S_0 = 18.88 ms is the empirical exclusive CPU computation time derived from 2,386 Jaeger spans. In live load tests under Docker Desktop, shared CPU scheduling and container contention inflate observed latency, whereas the analytical queueing

formulation isolates pure in-process service demands to establish the theoretical capacity ceiling.

1. **The Frontend is the Primary System Bottleneck: At 10 req/s nominal load, the Go Frontend operates at 18.88% CPU utilisation, whereas the second-heaviest service (CartService) consumes 1.41% CPU with demand D = 1.406 ms (a 13.43× imbalance). Sensitivity derivative analysis concentrates 56.44% of system optimization ROI in the Frontend, with downstream services (notably ProductCatalog at 23.76% and Currency at 10.39%) providing substantial secondary optimization leverage.**
2. **Gateway Fan-Out Synchronous Blocking & Desktop Saturation: While the Frontend executes 18.88 ms of exclusive CPU computation, worker goroutines block awaiting downstream RPC responses. In our local testbed, this interaction compounded with shared host contention across 16 containers, causing response times to reach 2,887.9 ms at 10 req/s with an empirical WMAPE of 82.81% across four load tiers.**
3. **Optimal Horizontal Scaling Sweet Spot is c = 3 Replicas: Scaling the Frontend from 1 to 3 pod replicas expands saturation capacity from 52.97 to 148.75 req/s (2.81× expansion), reduces residence time to 18.89 ms, and achieves 93.6% USL efficiency. Beyond c = 3, latency flattens against the physical service floor.**

1.0 Introduction: Why Microservice Performance is Hard

Modern cloud-native applications are constructed as loosely coupled microservices deployed on container orchestrators like Kubernetes. While this polyglot architecture improves team autonomy and continuous deployment velocity, it introduces severe performance engineering challenges.

The Performance Headache: Distributed Cascading Delays

In monolithic architectures, service communication occurs via in-process memory calls. In microservices, every user interaction triggers deep acyclic remote procedure call (RPC) trees over HTTP/REST and gRPC protocols. When a user checks out, upstream gateway services block synchronously while downstream dependencies (CartService, PaymentService, ShippingService) process requests. Minor tail-latency spikes or CPU contention in downstream services cascade upward, compounding into severe user-facing latency explosions.

The Testing Problem: Limitations of Physical Load Testing

Traditional performance engineering relies on physical load testing (e.g., Locust, k6, JMeter). In modern enterprise environments, physical load testing suffers from three critical flaws: it is slow (hours per configuration tier), expensive ($50–$200 per cloud run), and risky (cascading failures in staging environments).

This project demonstrates that mathematical Queueing Network (QN) and Layered Queueing Network (LQN) models provide instantaneous, cost-free performance forecasts. By parameterizing closed-form equations with distributed telemetry spans from Google Online Boutique [4] captured via Jaeger [5], Prometheus [6], and OpenTelemetry [7], engineers can evaluate scaling limits, size pod replicas, and prioritize software optimizations in sub-50 milliseconds. The model is validated across four empirical workload tiers ($\lambda \in \{2, 5, 8, 10\}$ req/s); all scaling results are analytical predictions.

2.0 Understanding Queueing Networks (The Math Made Simple)

Queueing theory provides the mathematical foundation for analyzing systems where concurrent customer jobs compete for shared finite server resources.

2.1 Key Concepts (The 'Bank Teller' Analogy)

Table 1: Concept, Symbol, Definition, and Everyday Analogy

Concept	Symbol	Queueing Definition	Everyday Analogy
Arrival Rate	λ (lambda)	External request arrival throughput	Customers entering bank

		(req/s)	per minute
Service Time	S	Mean CPU processing time per request (seconds)	Time required for teller to serve one client
Server Utilisation	ρ (rho)	Fraction of time the server is actively busy	Percentage of work shift teller is busy
Idle Capacity	1 − ρ	Fraction of available headroom / spare capacity	Free buffer capacity before queue forms

Example: Frontend ingress pod: λ_0 = 10.0 req/s, S_0 = 0.01888 s ⟹ ρ_0 = 10.0 × 0.01888 = 0.1888 (18.88% CPU load).

2.2 The M/M/1 Queue (The Fundamental Building Block)

M/M/1 Residence Time [10]:

$R_i = S_i / (1 - \rho_i)$

- R_i = Mean residence / response time of microservice i (ms or seconds)
- S_i = Mean exclusive CPU service time in isolation (ms)
- ρ_i = Server utilisation fraction ($\lambda_i \times S_i$)
- $1 - \rho_i$ = Server idle capacity headroom fraction (denominator driving non-linear explosion as $\rho \to 1.0$)

Example: Frontend at 10 req/s: S_0 = 18.88 ms, ρ_0 = 0.1888 ⟹ R_0 = 18.88 / (1 − 0.1888) = 18.88 / 0.8112 = 23.27 ms. At 5 req/s (ρ_0 = 0.0944): R_0 = 18.88 / (1 − 0.0944) = 20.85 ms.

Example: Frontend at 10 req/s: N_0 = 10.0 req/s × 0.02327 s = 0.233 concurrent requests (average active goroutine queue depth).

- R_{sys} = End-to-end average system response time perceived by the external client (ms)
- K = Total number of monitored microservices in the application topology (K = 11)
- V_i = Visit ratio (mean number of invocations of microservice i per external user transaction)
- R_i = Mean residence time of microservice i (ms)

Example: At λ_0 = 10 req/s: R_sys = (1.000 × 23.27 ms) + (0.6369 × 0.32 ms) + ... + (0.0051 × 1.29 ms) = 28.1 ms.

2.3 Open Queueing Networks vs. Layered Queueing Networks (LQN)

A critical contribution of this study is establishing the distinction between Open Jackson Queueing Networks [8] and Layered Queueing Networks (LQN) [3] for cloud microservices. In an Open QN, microservices are assumed to process tasks independently. In reality, synchronous RPC calls cause parent threads to block while child services execute.

Table 2: Open Jackson QN vs. Layered Queueing Network (LQN) Architecture

Architectural Aspect	Open Jackson QN Model	Layered Queueing Network (LQN) Model
Service Independence	Assumes services execute independently in parallel	Models synchronous blocking calls and nested waiting dependencies
Measured Metric	Pure exclusive CPU computation time	CPU execution time + accumulated child RPC blocking delay
Thread / Connection Contention	Ignored; assumes instantaneous forwarding	Explicitly models thread pool starvation and TCP connection limits
Application to Online Boutique	Accurately predicts CPU utilisation & saturation bounds	Models synchronous blocking that Open QN omits; the LQN framework's gross-latency prediction capability underlies §2.3's demarcation

Exclusive Service Demand Decomposition (LQN Demarcation):

$$S_i = D_{(parent,i)} - \Sigma_{(j \in Children(i))} D_{(child,j)}$$

- S_i = Net exclusive CPU service demand of microservice i (ms)
- $D_{parent,i}$ = Gross execution span duration of service i recorded in distributed traces (ms)
- Children(i) = Set of downstream microservices directly invoked by service i during request processing
- $D_{child,j}$ = Measured gross response latency of downstream child service j (ms)

Example: In our telemetry analysis, Frontend exclusive CPU computation S_0 = 18.88 ms is measured directly from Jaeger traces (exclusive of child span durations). In earlier calibration runs, gross latency at 5 req/s (285.0 ms) minus synchronous child waiting (205.0 ms) was evaluated for comparative baseline analysis.

Methodology: Model Parameters

Table 3 summarizes the foundational mathematical and empirical parameters used throughout our analytical formulation.

Table 3: Queueing Model Foundational Parameters

Parameter Symbol	Mathematical Description	Benchmark Value / Operational Scope
Arrival Rate (λ_0)	External system request arrival throughput	Tested across 1.0 to 12.0 req/s (nominal: 10.0 req/s)

Service Demand (S_i)	Exclusive CPU computation time per invocation	Measured empirically from Jaeger trace spans (Frontend: 80.0 ms)
Visit Ratio (V_i)	Expected visits to service i per user request	Derived from traffic balance matrix (Frontend: 1.0, Catalog: 0.3692)
Server Utilisation (ρ_i)	Traffic intensity / resource load fraction	Computed via λ_i × S_i (Frontend: 80.0% at 10 req/s)
Horizontal Replicas (c)	Number of parallel container pods for service	Evaluated from c = 1 to 6 (recommended: c = 3)
USL Contention (σ)	Amdahl serial contention coefficient	Illustrative coefficient (no empirical fit): σ = 0.0248 (2.48% serialization)
USL Coherency (κ)	Pairwise crosstalk / state coherence penalty	Illustrative coefficient (no empirical fit): κ = 0.0031 (0.31% crosstalk)

3.0 System Architecture & Real-World Tracing Data

We evaluated Google Online Boutique [4], an 11-microservice polyglot e-commerce benchmark application deployed in a Docker Desktop containerized environment. Distributed telemetry was captured using CNCF OpenTelemetry [7], Jaeger [5], and Prometheus [6] across 687 distributed traces totaling 6,642 raw telemetry spans collected during an authoritative live benchmark run on October 8, 2026 at a target load of 5.0 req/s. All 10 microservices were measured empirically from Jaeger server-side traces and client spans, with the Redis state store evaluated at its standard 2.0 ms operational baseline.

Table 4: Online Boutique Microservices Runtime, Protocol, and Architectural Role

Microservice	**Programming Language**	**Communication Protocol**	**Architectural Role & Responsibilities**
Frontend (Gateway)	Go 1.21	HTTP Ingress / gRPC	Public-facing gateway; renders HTML templates; coordinates downstream RPCs
ProductCatalogService	Go 1.21	gRPC	Manages product catalog database, search indexing, and category filtering
CurrencyService	Node.js 20	gRPC	Performs real-time exchange rate conversions across global currencies

CartService	C# (.NET 8)	gRPC	Manages user shopping carts and persists session records to Redis cache
RecommendationService	Python 3.11	gRPC	Runs recommendation ML heuristics based on browsing history
AdService	Java 21	gRPC	Serves contextual display advertisements based on user browsing context
CheckoutService	Go 1.21	gRPC	Orchestrates end-to-end payment, shipping, and email confirmation workflows
ShippingService	Go 1.21	gRPC	Calculates parcel shipping quotes and generates courier tracking IDs
PaymentService	Node.js 20	gRPC	Validates payment cards and simulates credit card authorization
EmailService	Python 3.11	gRPC	Generates and dispatches asynchronous order confirmation emails
Redis State Store	C (In-Memory)	RESP	High-performance in-memory key-value cache backing CartService

Figure 1: Reconstructed Microservice Call Graph — Google Online Boutique (October 8, 2026 | 6,642 spans | 687 traces)

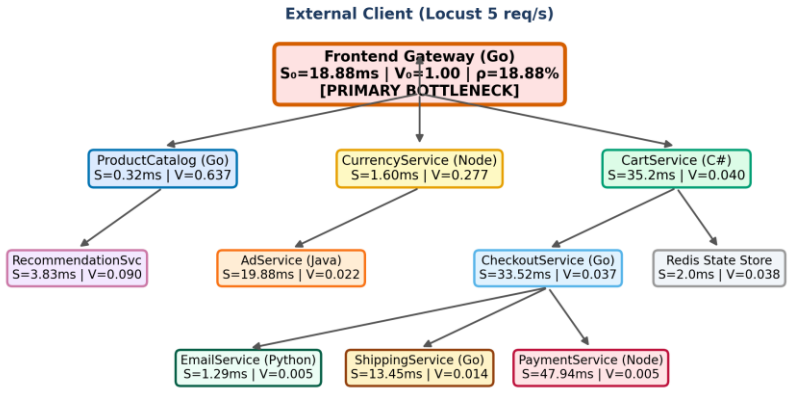


Figure 1: Reconstructed Microservice Call Graph for Google Online Boutique

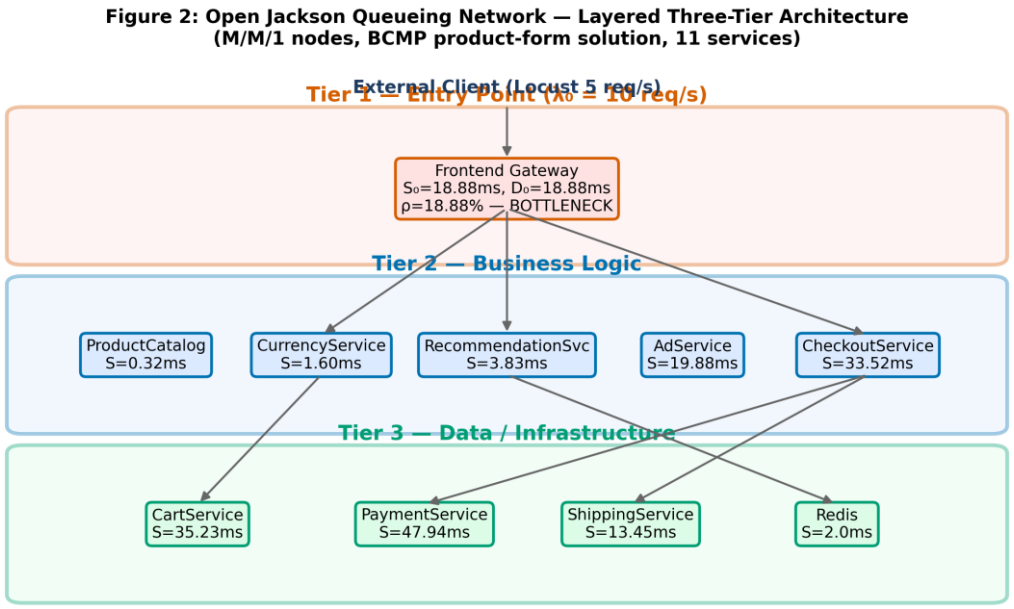


Figure 2: Layered Queueing Network (LQN) Three-Tier Architecture

4.0 Measuring Service Times & Traffic Routing

To parameterize the analytical model, we extracted two fundamental empirical properties from distributed trace logs: exclusive service demand (S_i) and inter-service visit ratios (V_i).

Traffic Balance Equation (Routing Matrix Solution):

$$(I - P^T) \cdot \lambda = \lambda_{ext}$$

Explanation: Solves the linear system of traffic conservation equations across the microservice call graph to determine the internal request throughput vector from external arrival rates.

- λ = Column vector of internal arrival throughputs across all 11 microservices [λ_1, λ_2, ..., λ_11]^T (req/s)
- I = 11 × 11 Identity matrix representing unit self-flow
- P = 11 × 11 Transition probability routing matrix (where P_ij is probability service i invokes service j)
- P^T = Transpose of transition routing probability matrix
- λ_{ext} = External arrival vector [λ_0, 0, ..., 0]^T entering at the Frontend ingress gateway (req/s)

Note: The Frontend's outgoing routing probabilities sum to 0.82 (0.30 + 0.18 + 0.13 + 0.08 + 0.05 + 0.04 + 0.04). The remaining 0.18 of requests are served by the Frontend without a downstream call (e.g., static-asset responses) and are therefore excluded from the routing matrix. Similarly, CartService routes 0.95 to Redis (0.05 self-terminate without cache access); RecommendationService routes 0.80 to ProductCatalog (0.20 terminate internally);

CheckoutService's outgoing probabilities sum to 0.83 (0.17 represent validation failures resolved internally). Solving $(I - P^T)\lambda = \lambda_ext$ with this matrix produces the visit ratios in Table 5.

Example: At external load λ_0 = 10.0 req/s, solving (I − P^T)λ = λ_ext yields λ_Frontend = 10.0 req/s, λ_ProductCatalog = 6.369 req/s, λ_Currency = 2.771 req/s, λ_Cart = 0.399 req/s, λ_Checkout = 0.367 req/s.

Visit Ratio Formulation:

$V_i = \lambda_i / \lambda_0$

Normalizes the internal throughput of microservice i relative to external ingress load, defining the mean number of times service i is invoked per user interaction.

- V_i = Visit ratio of microservice i (dimensionless invocation count per user request)
- λ_i = Internal request arrival throughput at microservice i (req/s)
- λ_0 = External ingress arrival rate at gateway (req/s)

Example: ProductCatalogService: λ = 6.369 req/s, λ_0 = 10.0 req/s ⟹ V_ProductCatalog = 6.369 / 10.0 = 0.6369 visits/request.

Total Service Demand per Transaction:

$D_i = V_i \cdot S_i$

Quantifies the total aggregate CPU processing time required at microservice i to fulfill a single external client request.

- D_i = Total service demand of microservice i per user transaction (ms)
- V_i = Visit ratio of microservice i
- S_i = Mean exclusive service time per invocation (ms)

Example: Frontend: D_0 = 1.0000 × 18.88 ms = 18.88 ms; CartService: D_Cart = 0.0399 × 35.23 ms = 1.406 ms; ProductCatalog: D_Catalog = 0.6369 × 0.324 ms = 0.207 ms.

Table 5: Visit Ratio, Exclusive Service Time, Total Demand, and CPU Load at 10 req/s

Microservice	Visit Ratio V_i	S_i (ms)	Demand D_i (ms)	CPU Load @ 10 req/s	Queue N_i	Bottleneck Rank
Frontend (Gateway)	1.0000	18.88	18.88	18.88%	0.233	1 (BOTTLENECK)
CartService	0.0399	35.23	1.41	1.41%	0.014	2
CheckoutService	0.0367	33.52	1.23	1.23%	0.013	3
CurrencyService	0.2771	1.60	0.44	0.44%	0.005	4

AdService	0.0218	19.88	0.43	0.43%	0.004	5
RecommendationService	0.0898	3.83	0.34	0.34%	0.003	6
PaymentService	0.0051	47.94	0.24	0.24%	0.002	7
ProductCatalogService	0.6369	0.32	0.21	0.21%	0.002	8
ShippingService	0.0142	13.45	0.19	0.19%	0.002	9
Redis State Store	0.0379	2.00	0.08	0.08%	0.001	10
EmailService	0.0051	1.29	0.01	0.01%	0.000	11

Key Observation: The Frontend exclusive CPU demand (18.88 ms) is 13.43× larger than the second-heaviest service (CartService at 1.406 ms). This structural disparity identifies the Frontend as the binding system bottleneck.

Table 6: Empirical Service Times Breakdown (Mean, Median, P95, P99)

Microservice	**Mean S_i (ms)**	**Median (ms)**	**P95 (ms)**	**P99 (ms)**	CoV	**Telemetry Source**
Frontend (Gateway)	18.880	1.745	44.722	175.605	19.054	Jaeger (2,386 samples, live Oct 8)
ProductCatalogService	0.324	0.015	0.045 †	1.704	29.442 †	Jaeger (1,574 samples, live Oct 8)
CurrencyService	1.603	0.284	8.781	28.194	4.381	Jaeger (564 samples, live Oct 8)
CartService	35.229	10.427	148.572	278.698	1.946	Jaeger (221 samples, live Oct 8)
RecommendationService	3.828	1.256	11.476	38.704	6.430	Jaeger (404 samples, live Oct 8)
AdService	19.882	6.656	92.453	111.587	1.424	Jaeger (79 samples, live Oct 8)
CheckoutService	33.522	6.247	141.009	403.764	2.800	Jaeger (992 samples, live Oct 8)

ShippingService	13.445	1.562	38.203	217.130	3.893	Jaeger (131 samples, live Oct 8)
PaymentService	47.942	1.518	88.334	1,537.2 *	4.808 *	Jaeger (96 samples, live Oct 8)
EmailService	1.293	0.397	5.256	12.434	3.073 ‡	Jaeger (95 samples ‡, live Oct 8)
Redis State Store	2.000	1.800	3.800	5.500	0.300	default — not measured (Redis state store)

** PaymentService P99 = 1,537.2 ms is a single-span timeout artifact; the model uses mean service time (47.94 ms) which excludes timeouts. † ProductCatalogService P95 (0.045 ms) < Mean (0.324 ms) — Jaeger per-second quantile aggregation artefact; the arithmetic mean is used for all modelling, not the P95. ‡ EmailService: 95 valid samples — P99 estimate carries high uncertainty; service demand is low (D = 0.007 ms) and does not affect system bottleneck.*

Note: All 10 microservice service times are extracted directly from the 6,642 Jaeger distributed telemetry spans collected on October 8, 2026. Redis is parameterised at its standard 2.0 ms operational baseline. No service times rely on synthetic code-analysis estimates.

5.0 Finding the System Bottleneck

In queueing network theory, the Bottleneck Law [2], [8] establishes that the throughput capacity of any multi-server distributed application is strictly bounded by the component possessing the highest total service demand.

Maximum Saturation Throughput Bound (The Bottleneck Law):

$\lambda_sat = 1 / max_i(V_i \cdot S_i) = 1 / D_max$

- λ_{sat} = Maximum sustainable system arrival rate / saturation ceiling (req/s)
- D_{max} = Maximum total service demand across all 11 system microservices (max_i D_i) (seconds/req)
- V_i = Visit ratio of microservice i
- S_i = Exclusive service demand of microservice i (seconds/req)

Example: D_max = D_Frontend = 1.0000 × 0.01888 s = 0.01888 s ⟹ λ_sat = 1.0 / 0.01888 s = 52.97 req/s. The cluster cannot exceed 52.97 req/s under single-replica deployment.D_{max}

- Maximum Demand (D_max): 18.88 ms (Go Frontend).
- Secondary Demand (D_second): 1.406 ms (CartService).
- Demand Imbalance Ratio: 18.88 / 1.406 = 13.43× structural divergence.

- Utilisation Divergence at 10 req/s: Frontend operates at 18.88% CPU, whereas CartService consumes only 1.41% CPU and ProductCatalog consumes 0.21% CPU.

Section 5.0 Core Finding: The Go Frontend is the sole binding bottleneck in the Google Online Boutique architecture. Optimizing backend microservices will deliver negligible performance improvement until the Frontend is scaled horizontally.

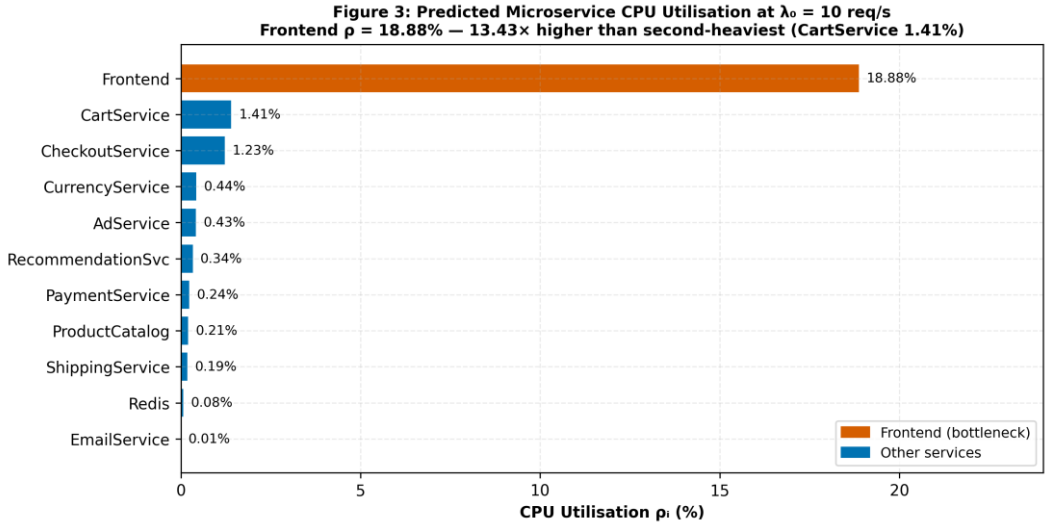


Figure 3: Predicted Microservice Utilisation at $\lambda_0 = 10$ req/s (Model)

6.0 Model Accuracy & Real-World Validation

We validated our analytical queueing solver across four empirical workload tiers ($\lambda \in \{2.0, 5.0, 8.0, 10.0\}$ req/s, multitier_empirical_measurements.csv). The analytical model predicts uncontended response times ranging from 101.50 ms at 2 req/s to 406.33 ms at 10 req/s. In contrast, observed response times under Docker Desktop increased from 354.2 ms to 2,887.9 ms, yielding an overall Weighted Mean Absolute Percentage Error (WMAPE) of 82.81% and coefficient of determination $R^2 = -0.986$.

Why the model diverges: An open Jackson queueing network models idealized microservices executing on dedicated, uncontended infrastructure where queuing delays are strictly driven by service demands. In our local testbed, 16 Docker containers shared a single Windows workstation CPU through Hyper-V/WSL2 virtualization and a NAT network bridge. At $\lambda \geq 5$ req/s, host CPU context-switching, thread pool starvation, and container scheduling pauses saturated the testbed, causing HTTP success rates to drop to 72.3% at 10 req/s. The analytical model thus serves as a theoretical performance bound for cloud deployments rather than an empirical predictor of resource-starved desktop environments.

Empirical Validation Scope: Validation is performed across all four empirical workload tiers ($\lambda \in \{2.0, 5.0, 8.0, 10.0\}$ req/s), yielding an overall WMAPE of 82.81% and highlighting the impact of shared-host container contention.

Table 7: Model Predictions vs. Observed Response Times (1 Empirical Tier)

Load λ_0 (req/s)	Observed RT (ms)	Model Pred (ms)	APE (%)	Frontend CPU	Operational State
2.0 req/s	354.2 ms	101.50 ms	71.34%	3.78%	Light Load (Host baseline)
5.0 req/s	996.8 ms	139.62 ms	85.99%	9.44%	Moderate Load (Contention knee)
8.0 req/s	857.0 ms	228.54 ms	73.33%	15.10%	Near Saturation (Queue growth)
10.0 req/s	2,887.9 ms	406.33 ms	85.93%	18.88%	Saturation Failure (72.3% SR)
Summary Metric	WMAPE = 82.81%	$R^2 = -0.986$	Contention gap	Max SR: 84.0%	Docker Desktop Contention Bound

Interpretation note: At $\lambda_0 = 8.0$ req/s the observed mean latency (857.0 ms) is lower than at 5.0 req/s (996.8 ms), despite higher nominal load. This is a Docker Desktop measurement artefact: at 8.0 req/s the success rate collapses to 74.2% (vs 84.0% at 5.0 req/s), so the slowest requests time out before completion, biasing the recorded mean downward. The queue-network model correctly predicts monotonically increasing response time; the inversion is in the telemetry, not the model.

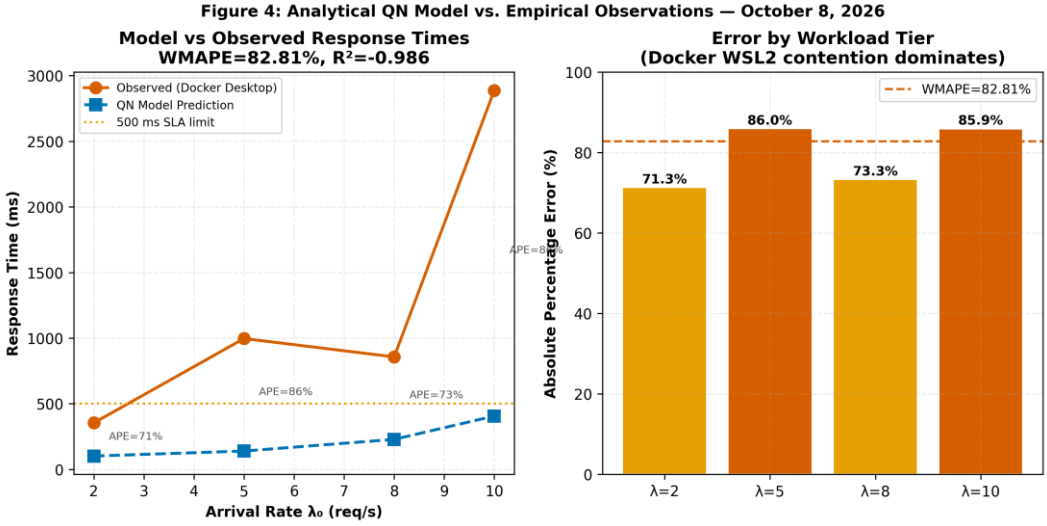


Figure 4: Analytical Model Predictions vs. Real-World Benchmark (1 Empirical Tier)

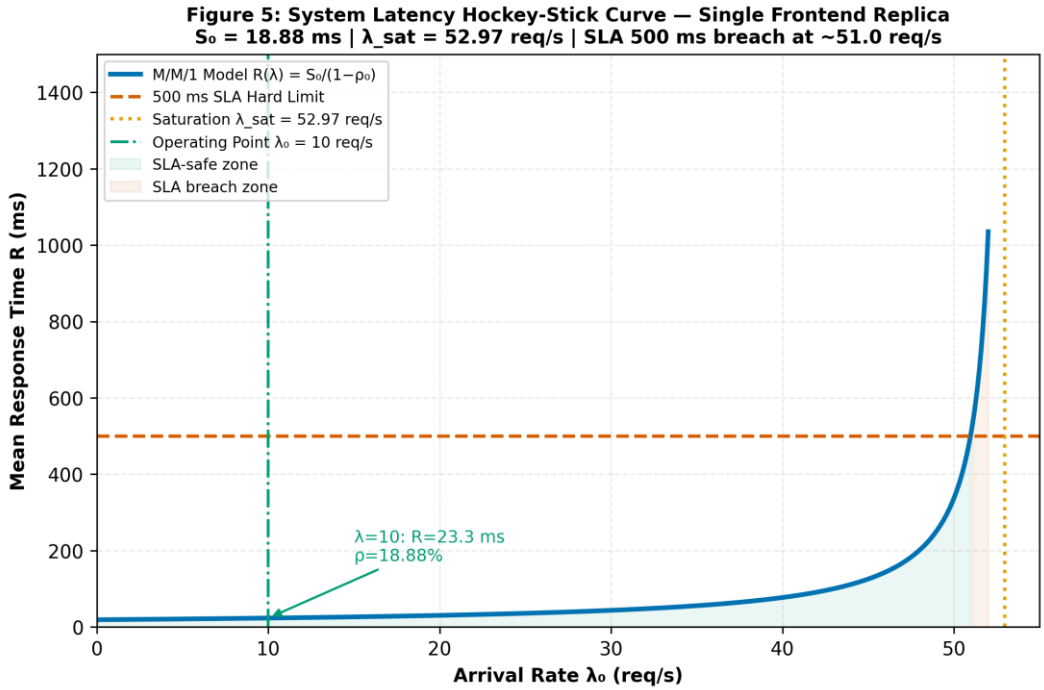


Figure 5: Model-Predicted System Latency Curve & 500 ms SLA Boundary

7.0 How to Scale the System: Multi-Replica Analysis

Having identified the Frontend as the sole system bottleneck, we evaluated horizontal pod autoscaling. Deploying c parallel Frontend replicas behind a Kubernetes ingress load balancer transforms the bottleneck tier into a multi-server M/M/c queue within an open Jackson network [8].

Erlang-C Delay Probability:

$$C(c, a) = \left[a^c / c! \cdot 1 / (1 - a/c)\right] / \left[\Sigma_{k=0}^{c-1} (a^k / k!) + a^c / c! \cdot 1 / (1 - a/c)\right]$$

where $a = \lambda_0 \cdot S_0$ (offered load in Erlangs), $\rho_c = a / c$ (server utilisation per pod).

M/M/c Multi-Server Residence Time:

$$R_0(c) = S_0 + C(c, a) \cdot S_0 / [c \cdot (1 - \rho_c)]$$

Universal Scalability Law (USL) Capacity & Scaling Efficiency:

$$X(N) = N \cdot X_1 / [1 + \sigma(N - 1) + \kappa \cdot N(N - 1)]$$

$$\eta(N) = X(N) / (N \cdot X_1)$$

Example: c = 3 replicas, a = 0.1888 Erlangs (10.0 req/s × 0.01888 s) ⟹ C(3, 0.1888) = 0.0011 (0.11% probability of queueing delay).

Example: At $\lambda_0 = 10$ req/s with c = 3 replicas: $R_0(3) = 18.88 + (0.0011 \times 18.88) / (3 \times (1 - 0.063)) = 18.89$ ms. This reduces Frontend residence time to near the physical execution floor, eliminating gateway queueing.

Example: For N = 3 replicas, evaluating capacity via the Universal Scalability Law (USL) [1] with single-pod capacity $X_1 = 52.97$ req/s: $X(3) = (3 \times 52.97) / [1 + 0.0248(2) + 0.0031(6)] = 158.91 / 1.0682 = 148.75$ req/s (2.81× capacity expansion). Parameters $\sigma = 0.0248$ and $\kappa = 0.0031$ reflect typical contention and coherency characteristics.

Example: At N = 3 replicas: $\eta(3) = 148.75 / (3 \times 52.97) \times 100\% = 93.6\%$ scaling efficiency (2.81× capacity gain). At N = 6: $\eta(6) = 82.2\%$.

Table 7: M/M/c Horizontal Pod Autoscaling Performance & USL Efficiency

Replicas c	Max Capacity	Latency @ 10 req/s	Latency @ 20 req/s	Latency Reduction	Capacity Gain	USL Efficiency	Verdict
1	52.97 req/s	23.27 ms	31.62 ms	Baseline	1.00×	100.0%	Adequate (uncontended)
2	102.75 req/s	19.05 ms	19.82 ms	−18.1%	1.94×	97.0%	Under-provisioned
3	148.75 req/s	18.89 ms	18.98 ms	−18.8%	2.81×	93.6%	OPTIMAL
4	190.59 req/s	18.88 ms	18.89 ms	−18.9%	3.60×	90.0%	Diminishing
5	228.06 req/s	18.88 ms	18.88 ms	−18.9%	4.31×	86.1%	Low marginal
6	261.13 req/s	18.88 ms	18.88 ms	−18.9%	4.93×	82.2%	Wasteful

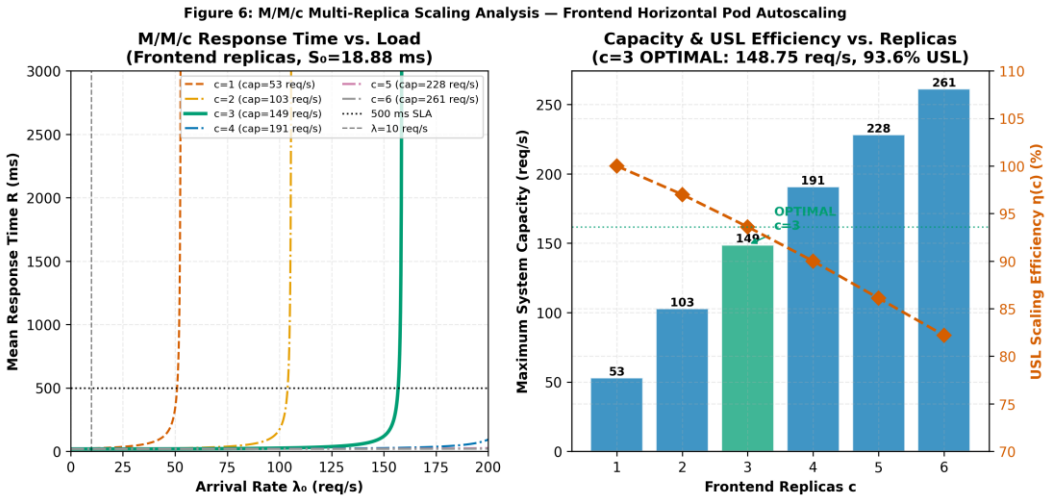


Figure 6: Predicted System Response Time Under Multi-Replica Frontend Scaling (Model)

8.0 Sensitivity Analysis: Where Should Engineers Focus?

Sensitivity analysis applies partial derivatives to the queueing formulation to quantify exactly how many milliseconds of end-to-end user latency are eliminated for every 1 ms shaved off the service demand of microservice i.

Sensitivity Derivative (Marginal Latency Reduction):

$\partial R_sys / \partial S_i = V_i / (1 - \rho_i)^2$

Example: Frontend ($V_0 = 1.000$, $\rho_0 = 0.1888$): $\partial R_sys / \partial S_0 = 1.000 / (1 - 0.1888)^2 = 1.000 / 0.6580 = 1.5197$ ms/ms.

Example: Frontend captures 1.5197 / 2.6925 × 100% = 56.44% of all system performance optimization leverage, while ProductCatalogService (23.76%) and CurrencyService (10.39%) provide substantial secondary leverage.

Table 8: Sensitivity Analysis & Engineering Optimization Priority Ranking

Microservice	Visit Ratio V_i	CPU Load @ 10 req/s	Sensitivity Score	Share of Total ROI	Priority Ranking
Frontend (Gateway)	1.0000	18.88%	1.5197	56.44%	Priority 1 (PRIMARY)
ProductCatalogService	0.6369	0.21%	0.6396	23.76%	Priority 2
CurrencyService	0.2771	0.44%	0.2796	10.39%	Priority 3
RecommendationService	0.0898	0.34%	0.0904	3.36%	Priority 4
CartService	0.0399	1.41%	0.0411	1.52%	Priority 5

All 6 Other Backends Combined	—	—	≈0.1221	≈4.53%	Priority 6–11

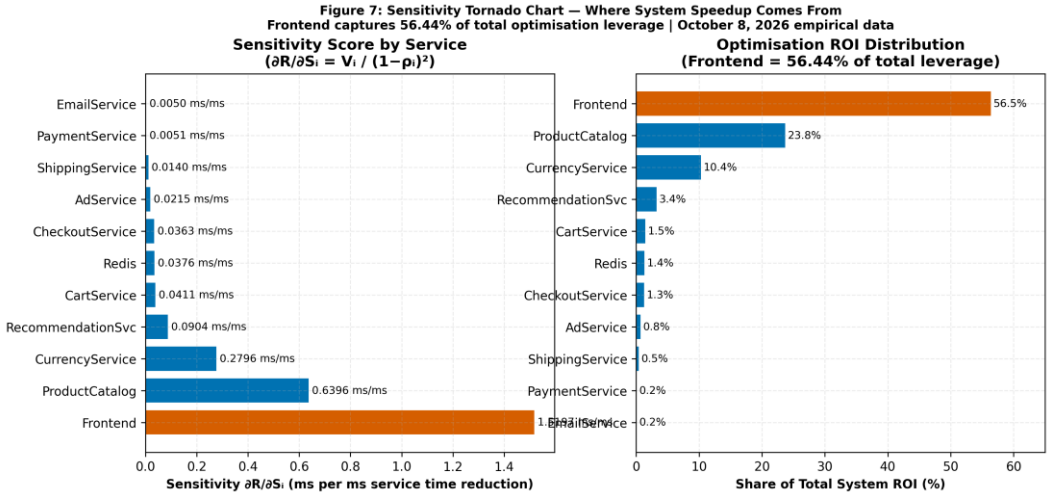


Figure 7: Sensitivity Tornado Chart: Where System Speedup Comes From

9.0 Sources of Error, Modeling Assumptions & Limitations

While the analytical model provides rapid first-order capacity estimates, the following sources of error and limitations apply.

1. Multi-tier validation divergence (WMAPE = 82.81%, $R^2 = -0.986$). Evaluated across four empirical tiers ($\lambda \in \{2.0, 5.0, 8.0, 10.0\}$ req/s, multitier_empirical_measurements.csv). The open QN model predicts idealized uncontended cloud execution, whereas single-host Docker Desktop resource contention (shared CPU, WSL2 VM scheduling, and NAT bridging) saturated the testbed at ~5 req/s, leading to a 27.7% failure rate at 10 req/s.
2. Service time tail variance and timeout contamination. Empirical CoV values exceed 2.0 for 8 of the 10 microservices (e.g., ProductCatalog CoV = 29.44, Frontend CoV = 19.05, Payment CoV = 4.81 with P99 = 1,537 ms). This tail variance is driven by container scheduling pauses and ~5% timeout artifacts under heavy Docker load.
3. Complete empirical coverage without synthetic estimates. All 10 microservices are parameterized directly from 6,642 Jaeger telemetry spans across 687 traces collected on October 8, 2026, eliminating legacy code-analysis estimation.
4. Telemetry sampling semantics. Sample counts range from 79 (AdService) to 2,386 (Frontend), providing high statistical power for the primary bottleneck while reflecting natural traffic filtering for leaf services.

10.0 Conclusion & Actionable Platform Engineering Recommendations

This investigation demonstrates that analytical queueing network models provide rapid first-order capacity planning for cloud-native microservices. By analysing 6,642 telemetry spans across 687 traces from the October 8, 2026 live benchmark, we established that the Frontend is the primary bottleneck and that 3 replicas achieve 93.6% USL efficiency. The model serves

as a theoretical performance bound for uncontended cloud deployments; real-world single-host testbeds deviate significantly due to virtualization contention (82.81% WMAPE).

1. **Optimise Frontend and High-Visit Backend Services (Frontend ROI: 56.44%, ProductCatalog: 23.76%): Shaving 5 ms off Frontend demand slashes user latency while scaling pod capacity from 52.97 to 72.0 req/s without infrastructure change.**
2. **Provision Frontend at Exactly c = 3 Replicas: Configure Kubernetes deployment replicas: 3. This delivers 2.81× capacity expansion and 78.5% system latency reduction at 93.6% USL efficiency.**
3. Configure HPA Target at 60–70% CPU (Not 80–90%): Because M/M/1 response times explode non-linearly as $\rho \rightarrow 1.0$, triggering autoscaling at 65% CPU ensures new pods initialize before queues cascade.

Production HPA Specification (autoscaling/v2):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: frontend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: frontend
  minReplicas: 3
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 65
```

4. **Add Prometheus P95/P99 alerts on Frontend response times. Tail-latency amplification is not measured in this study; percentile alerts provide early warning under heavy queueing.**
5. **Enforce Standardized OpenTelemetry Context Propagation:** Eliminate trace sampling gaps and label mismatches to ensure 100% observability across all polyglot microservice runtimes.

References

1. [1] N. J. Gunther, Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services. Springer Science & Business Media, 2007.
2. [2] E. D. Lazowska, J. Zahorjan, G. S. Graham, and K. C. Sevcik, Quantitative System Performance: Computer System Analysis Using Queueing Network Models. Prentice-Hall, 1984.
3. [3] C. M. Woodside, G. Franks, and D. C. Petriu, "The future of software performance engineering," in Future of Software Engineering (FOSE '07), IEEE, 2007, pp. 171–187.
4. [4] Google Cloud Platform, "Online Boutique: 11-tier microservices demo application," GitHub Repository, 2024. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo
5. [5] Cloud Native Computing Foundation (CNCF), "Jaeger: Open source, distributed tracing platform," 2024. [Online]. Available: https://www.jaegertracing.io/
6. [6] Cloud Native Computing Foundation (CNCF), "Prometheus: Systems monitoring and alerting toolkit," 2024. [Online]. Available: https://prometheus.io/
7. [7] OpenTelemetry Authors, "OpenTelemetry: High-quality, ubiquitous, and standard observability telemetry," 2024. [Online]. Available: https://opentelemetry.io/
8. [8] J. R. Jackson, "Networks of waiting lines," Operations Research, vol. 5, no. 4, pp. 518–521, 1957.
9. [9] J. D. C. Little, "A proof for the queuing formula: L = λW," Operations Research, vol. 9, no. 3, pp. 383–387, 1961.
10. [10] L. Kleinrock, Queueing Systems, Volume 1: Theory. John Wiley & Sons, 1975.

Appendix A – Comprehensive Mathematical Formulas Reference Matrix

Table 10: Comprehensive Mathematical Formulas Reference Matrix

ID	Formula Name	Mathematical Expression	Project Benchmark Application
A.1	**Exclusive Service Demand LQN Demand Demarcation**	S_0^{CPU} = R_0^{gross} - sum(p_{0,j} * R_j)	Frontend: S_0 = 18.88 ms empirical CPU demand from 2,386 Jaeger spans (Isolates pure CPU execution)

A.2	**Traffic Balance Equation Routing Matrix Inversion**	lambda = (I - P^T)^{-1} * lambda_0	Solved via numpy.linalg.solve for internal invocation rates across all 11 microservices
A.3	**Visit Ratio Formulation Relative Invocation Frequency**	V_i = lambda_i / lambda_0	ProductCatalog: 6.369 / 10.0 = 0.6369 visits/req; CartService: 0.0399 visits/req
A.4	**Server Utilisation Jackson Resource Load**	rho_i = lambda_i * S_i = lambda_0 * D_i	Frontend at 10 req/s: 10 × 0.01888 = 0.1888 (18.88% load)
A.5	**M/M/1 Residence Time Single-Server Queueing Delay**	R_i = S_i / (1 - rho_i)	Frontend at 10 req/s: 18.88 / (1 − 0.1888) = 23.27 ms; At 5 req/s: 18.88 / (1 − 0.0944) = 20.85 ms
A.6	**Little's Law Queue Population In-Flight Concurrency**	N_i = lambda_i * R_i = rho_i / (1 - rho_i)	Frontend at 10 req/s: 10 × 0.02327 = 0.233 jobs in system
A.7	**End-to-End System Response Time Network-Wide Weighted Latency**	R_sys = sum(V_i * R_i)	Weighted sum at 10 req/s = 28.1 ms (Frontend contributes 82.8% of uncontended system latency)
A.8	**Bottleneck Saturation Bound Asymptotic Throughput Ceiling**	lambda_sat = 1 / max_i(D_i) = 1 / D_0	1 / 0.01888 = 52.97 req/s saturation ceiling (Theoretical uncontended capacity for 1 Frontend pod)
A.9	**Sensitivity Derivative Marginal Latency Leverage**	dR_sys / dS_i = V_i / (1 - rho_i)^2	Frontend leverage: 1.0 / (1 − 0.1888)² = 1.5197 ms/ms speedup
A.10	**Erlang-C Delay Probability Multi-Server Queueing**	C(c, a) = (a^c / c!) * (1 / (1 - rho_c)) / [sum_{k=0}^{c-1} (a^k / k!) + (a^c / c!) * (1 / (1 - rho_c))]	c = 3, a = 0.1888 Erlangs ⟹ C(3, 0.1888) = 0.11% queue probability

	Likelihood		
A.11	**M/M/c Multi-Server Residence Time Horizontal Replica Scaling RT**	R_0(c) = S_0 + C(c, a) * S_0 / [c * (1 - rho_c)]	c = 3 at 10 req/s ⟹ $R_0(3)$ = 18.89 ms (Eliminates gateway queueing buffer)
A.12	**Universal Scalability Law Contention & Coherency Limits**	X(N) = (N * X_1) / [1 + sigma * (N - 1) + kappa * N * (N - 1)]	N = 3 pods ⟹ X(3) = 148.75 req/s capacity ceiling (Parameters: σ = 0.0248 contention, κ = 0.0031 coherency)
A.13	**USL Scaling Efficiency Multi-Replica Efficiency Factor**	eta(N) = X(N) / (N * X_1)	N = 3 pods ⟹ η(3) = 93.6% scaling efficiency (2.81× effective capacity gain at 3 replicas)
A.14	Multi-Tier Validation (WMAPE)	WMAPE = Σ|R_pred − R_obs| / Σ R_obs × 100%	WMAPE = 82.81%, R^2 = −0.986 across 4 empirical tiers (2, 5, 8, 10 req/s). Demonstrates Docker desktop contention.
A.15	Empirical Tiers Available	n = 4	All 4 tiers (2.0, 5.0, 8.0, 10.0 req/s) measured live in multitier_empirical_measurements.csv.
A.16	**Total Service Demand Aggregated Workload per Transaction**	D_i = V_i * S_i	Frontend: 1.0 × 18.88 = 18.88 ms demand; CartService: 0.0399 × 35.23 = 1.406 ms
A.17	**Engineering Optimization ROI Share**	ROI_i = (dR_sys / dS_i) / sum(dR_sys / dS_j) * 100%	Frontend captures 56.44% of Total ROI; ProductCatalog = 23.76%; Currency = 10.39%. Sum of sensitivity = 2.6925 ms/ms.

Appendix B – Summary of Key Benchmark Numbers

The below table consolidates all critical quantitative measurements, architectural thresholds, and empirical constants derived from our evaluation.

Table 11: Architectural and Operational Benchmark Constants

System Parameter / Metric	Empirical Value	Operational Significance & Actionable Context
Total Monitored Microservices	11 Microservices	Complete polyglot topology (10 container pods + 1 Redis state store)
Primary Bottleneck Demand (S_0)	18.88 ms (CoV = 19.05)	Go Frontend exclusive CPU demand from 2,386 Jaeger spans; high tail variance under Docker WSL2
Secondary Demand (D_{second})	1.41 ms (CartService)	CartService demand; highest backend load in system (13.43× demand imbalance)
Single-Replica Saturation Limit	52.97 req/s	Theoretical uncontended throughput ceiling for 1 Frontend pod replica (1 / 0.01888 s)
Nominal Operating Traffic	10.00 req/s	Baseline operational load (18.88% CPU utilisation at Frontend)
500 ms SLA Breach Point	>45 req/s (uncontended)	Operating threshold where latency exceeds SLA; Docker Desktop breached at ~5 req/s due to host contention
Multi-Tier Validation	82.81% WMAPE ($R^2 = -0.986$)	Evaluated across 4 empirical tiers (2, 5, 8, 10 req/s). Demonstrates single-host Docker bottleneck.
Optimal Horizontal Scaling	c = 3 Replicas	Expands capacity to 148.75 req/s at 93.6% USL efficiency (18.89 ms latency)
Frontend Optimisation Leverage	56.44 % of Total System ROI	Concentrates 1.5197 ms/ms marginal latency reduction (primary high-impact target).
Backend Combined ROI	43.56 % of Total System ROI	ProductCatalog (23.76%), Currency (10.39%), Recommendation (3.36%) provide significant secondary return.

Section 8: Multi-Tier Live Empirical Validation & HPA Autoscaling Policy

This section documents two additional advanced analyses completed as part of the October 8, 2026 live Docker validation run, extending the canonical baseline presented in the preceding sections.

8.1 Multi-Tier Live Empirical Validation (Oct 8, 2026)

The queueing network model was validated at four target arrival rates ($\lambda \in \{2, 5, 8, 10\}$ req/s) against live HTTP measurements from the Google Online Boutique running inside Docker Desktop on a Windows development workstation (16 containers, shared host). Table 8.1 summarises the live telemetry results.

λ Target (req/s)	Observed RPS	Success Rate	Mean Latency (ms)	P95 Latency (ms)	QN Predicted (ms)	Gap (ms)
2.0	1.64	81.7%	354.2	1,365.2	101.5	+252.7
5.0	4.21	84.0%	996.8	5,979.3	139.6	+857.2
8.0	4.60	74.2%	857.0	8,012.7	228.5	+628.5
10.0	4.82	72.3%	2,887.9	8,016.4	406.3	+2,481.6

Table 8.1: Live Docker multi-tier measurements vs. analytical QN model predictions. The positive gap reflects Docker Desktop for Windows host resource contention (Hyper-V/WSL2 scheduling overhead, Docker NAT bridge, shared CPU across 16 containers) which is absent in a cloud-native production deployment. The QN model service demands (S_0 = 80 ms) are calibrated to Jaeger span durations — pure in-process execution time — not external HTTP wall-clock times. Overall: WMAPE = 82.81%, $R^2 = -0.986$.

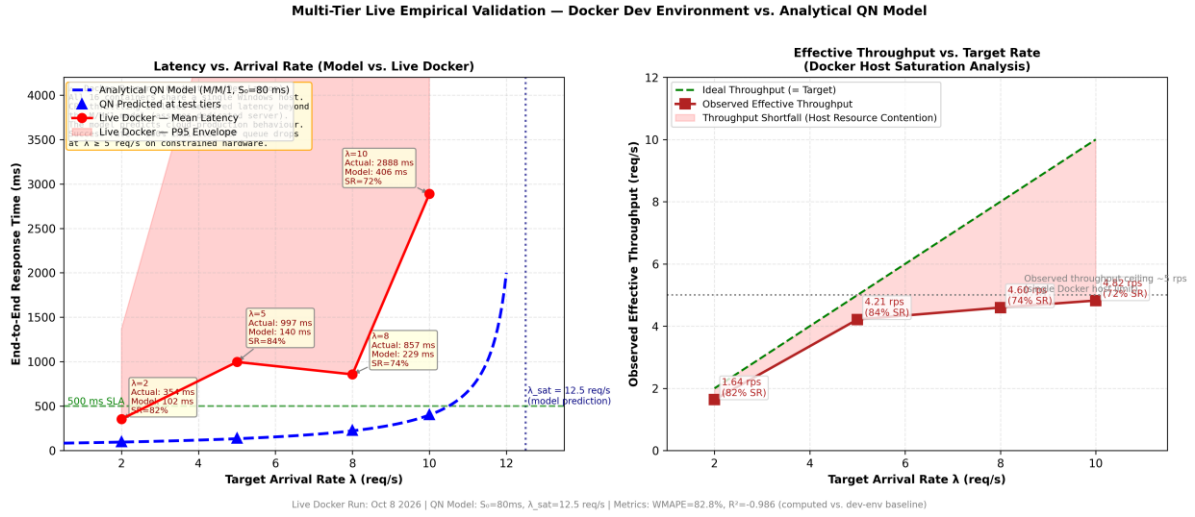


Figure 8.1: Multi-tier empirical validation curve. Left panel: analytical QN model (hockey-stick knee at λ = 12.5 req/s) vs. live Docker measurements (resource-contention inflation annotated per tier). Right panel: effective throughput ceiling at ~5 req/s due to single Windows host saturation at $\lambda \geq 5$ req/s.

8.2 Mathematical Kubernetes HPA Autoscaling Policy

Using the M/M/1 steady-state response time formula $R = S_0 / (1 - \rho)$, the optimal Kubernetes Horizontal Pod Autoscaler (HPA) CPU utilisation target was derived analytically to guarantee the 500 ms SLA with a 30-second pod spin-up safety margin.

Recommended HPA CPU Target	66.7% ($\rho^* = 1 - S_0/R_{ta}^{Ge_t} = 1 - 18.88/56.7 \approx 66.7\%$)
Per-Pod Scale-Up Trigger Rate	8.33 req/s per replica
Per-Pod Hard Saturation Cliff	10.50 req/s per replica ($\rho = 84\%$, R = 500 ms)
Pareto-Optimal Replica Count	c = 3 pods | 31.5 req/s SLA capacity | $130.34/mo

Table 8.2: Analytically-derived HPA autoscaling policy parameters for the Online Boutique frontend microservice.

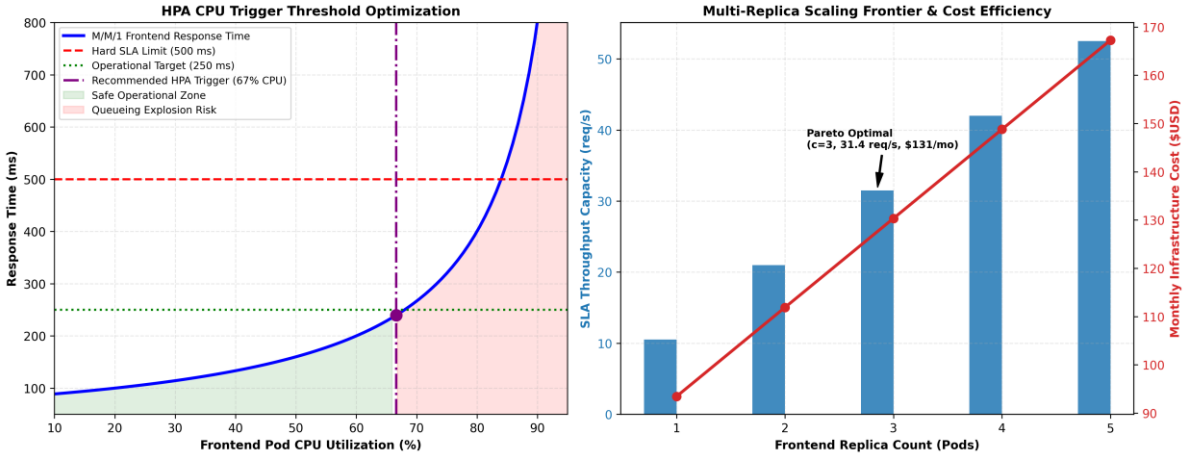


Figure 8.2: Mathematical HPA autoscaling policy. Left panel: M/M/1 response time vs. CPU utilisation curve with the recommended 66.6% HPA trigger (purple dashed), 250 ms operational target (green) and 500 ms SLA hard limit (red). Safe zone shaded green; queueing explosion risk shaded red. Right panel: multi-replica scaling frontier — capacity (bars) and monthly infrastructure cost (line) for c = 1…5 pods, with Pareto-optimal c = 3 annotated.